

COURSE SETUP & PRODUCTION MINDSET

Orientation

No code changes in this lecture.

Welcome: Market-Ready Engineer

أهلاً بك: من متابع شروحات إلى مهندس جاهز لسوق العمل

LEARNING OUTCOME

Understand the difference between following tutorials and making engineering decisions that can survive real users, deployments, and failures.

The gap is not “more tutorials.”

The gap appears when the lesson path disappears and the product starts behaving like a real system.

Code can work in a lesson and still fail as a product decision

REQUIREMENT CHANGES

The path is no longer fixed

You must decide what to build, what to defer, and what “done” means.

UNKNOWN FAILURE

The error is not in the video

You need a debugging process instead of a copied fix.

DEPLOYMENT

Localhost stops being enough

Builds, domains, cookies, network calls, and environment configuration become real.

Course promise: practical evidence - a complete product, a repeatable workflow, and language for explaining your engineering choices. **Not** a guarantee of employment.

Tutorial follower vs. market-ready engineer

Tutorial-following is a useful stage. Market readiness shows up when you can operate without a fixed path.

TUTORIAL MODE



Repeat a known sequence

Can follow steps when the lesson defines the decisions and the expected result.

- Works well while the path is known
- Can reproduce a demonstrated feature
- Gets stuck when requirements or errors are unfamiliar
- Often stops at “it works on my machine”

ENGINEERING MODE



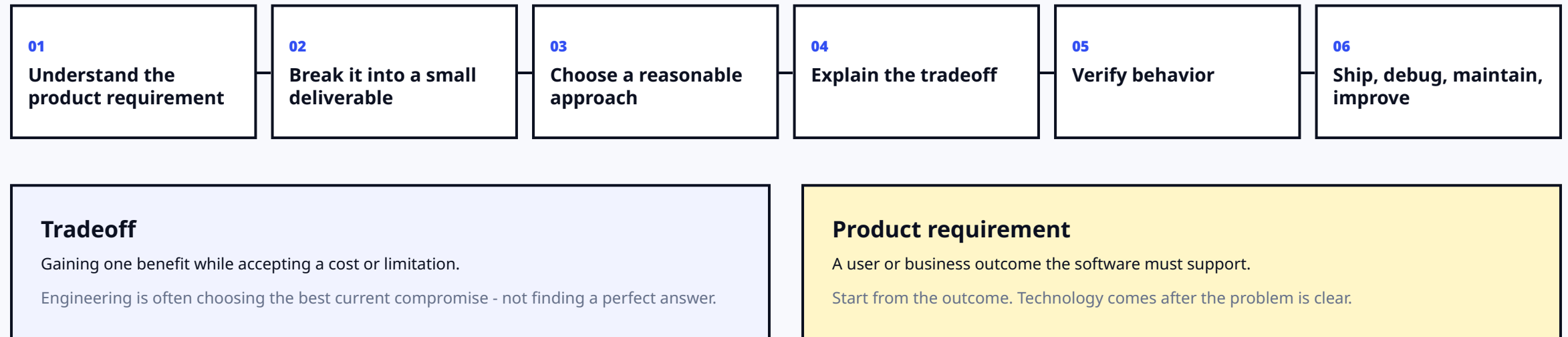
Turn a problem into a shippable step

Can choose a reasonable approach, explain the tradeoff, verify behavior, and improve a deployed application.

- Breaks requirements into small deliverables
- Makes tradeoffs explicit
- Verifies instead of assuming
- Ships, debugs, maintains, and improves

Market-ready is a repeatable capability loop

It does not mean memorizing every technology. It means turning an unstructured problem into something buildable and testable.



Production-first does not mean maximum complexity. The course introduces complexity only when the product earns it.

Same job-application form. Different engineering depth.

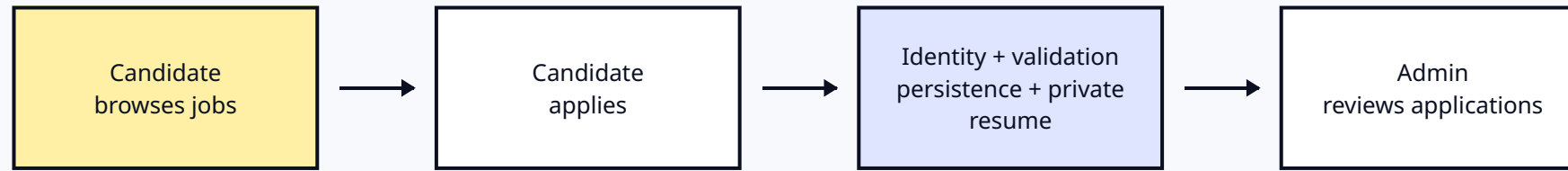
Both modes can start with the same UI. The difference becomes visible after the candidate presses Submit.

Tutorial mode	Engineering mode
Styled inputs	Server-side validation
Console log on submit	Signed-in candidate identified on the server
-	Application persisted
-	Resume stored privately
-	Failure handled without losing the submission
-	Flow verified on the deployed URL

- Who is allowed?
- Where does data live?
- What if a dependency fails?
- Can we debug production?
- Is the smallest solution safe enough now?

wazifa.app is the laboratory

Students practice engineering decisions through one product that evolves over the course.



BOUNDARIES MATTER

Candidate access

Browsing can be public. Applying eventually requires identity and server-side rules.

Admin access

Admin review exists, but candidates must never gain that access.

AI assistance

Useful only when an AI failure cannot silently destroy a candidate application.

Public and admin experiences share one codebase while preserving clear boundaries. Day 0 previews this destination; it does not teach the implementation yet.

The stack is a roadmap, not a prerequisite list

Each tool enters later when it solves a clear product problem. Lecture 1 introduces none of these as implementation work.

Next.js 16 Application framework Roadmap	React 19 UI foundation Roadmap	TypeScript Typed application code Roadmap	Tailwind / shadcn UI system Roadmap
MongoDB / Mongoose Persistence Roadmap	Zod Validation Roadmap	NextAuth Authentication Roadmap	DigitalOcean Deployment infrastructure Roadmap
Cloudflare Domain / edge services Roadmap	OpenAI AI-assisted screening Roadmap	Rule for the whole course Technology enters because the product needs it - not because the syllabus wants to show more tools.	

You need a foundation - not production experience

This course is for developers ready to move from isolated tutorials toward end-to-end product ownership.

YOU SHOULD KNOW

HTML & CSS

Basic page structure and styling.

Modern JavaScript

Functions, objects, arrays, async basics.

React fundamentals

Components, props, state, events.

Git + command line

Basic repository and terminal usage.

NOT REQUIRED YET

Next.js production deployments

Authentication

MongoDB

File storage

Docker

OpenAI APIs

DNS

Good fit: junior/intermediate React developers, frontend developers moving full-stack, developers who have built tutorials but not owned deployment, and teams/instructors who want a production-minded project reference.

Three sources keep learning aligned with the real project

The course is not just videos. The codebase and documentation work together so you can see reasoning, state, and handoff.

1

Lecture

`docs/day-*/lecture-*.md`

Reasoning and the next incremental change.

2

Repository

Version-controlled project and history.

The working source of truth at each stage.

3

Day documentation

Decisions, narrative, verification, and handoff.

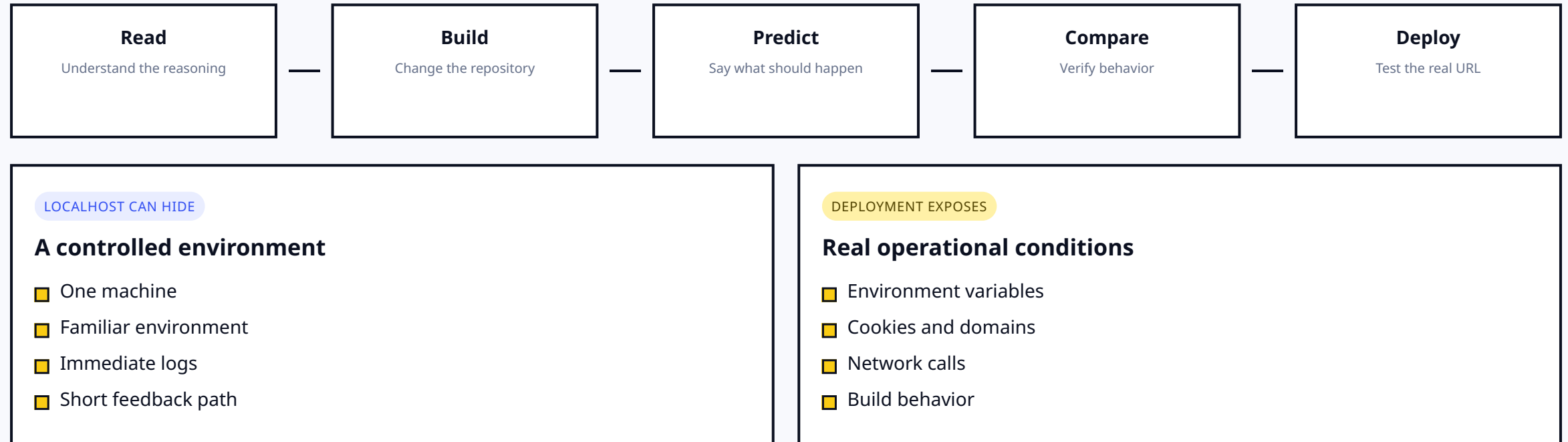
Use the day index to understand progression.

Start in the repository: skim `AGENTS.md` for the current architecture snapshot, then browse `docs/index.md` for the day-by-day lecture index.

Source of truth: the place treated as the current accurate state. Compare behavior and current repository state - not only copied file text.

The learning routine: read, build, predict, compare, deploy

Students are expected to reason about behavior before and after changing the application.



Before moving on, can you explain these without code?

Lecture 1 is complete when the mental model is clear - not when a file changes.

- ☐ Explain “market-ready” without treating it as a job guarantee.
- ☐ Name the prerequisites and what prior experience is not required.
- ☐ State the learning routine: read -> build -> predict -> compare -> deploy.
- ☐ Contrast tutorial-following with engineering-oriented product work.
- ☐ Name the three sources: lectures, repository, and day docs.
- ☐ Preview wazifa.app flows at a high level without implementation details.

SCOPE GUARDRAIL

Do not teach the implementation yet.

App Router internals, database schema, auth callbacks, storage signing, OpenAI API details, and “final architecture” claims are intentionally out of scope.

Lecture 1 takeaway

The course is designed to build evidence of engineering behavior through one deployed product.

01

Market-ready is shown through decisions and delivery.

02

You need a reliable way to learn and ship - not every technology.

03

wazifa.app is the laboratory for the course.

04

Complexity enters when the product earns it.

05

The promise is practice and evidence, not guaranteed employment.

NEXT LECTURE

Lecture 2 - What is Production Ready?

Turn a vague phrase into practical, repeatable engineering habits.

